

W13D4 - XSS e SQLi

metasploitable - exploit

XSS & SQLi

Danilo 'danix' Macrì - mia.mail@mio.sito

Scan started on 08/10/2025 - duration (days): 1

Contents

Sommario	1
Scope	1
Tecnologie utilizzate	1
Exploit	2
XSS - Security Low done	2
XSS - Security Medium done	2
XSS - Security High done	3

Sommario

Oggi eseguiremo un pentest mirato alle vulnerabilità di cross-site scripting (XSS) e SQL injection (SQLi) sulla nostra macchina Metasploitable2.

[!NOTE]

L'attacco di cross-site scripting consiste nell'andare a inserire codice malevolo in campi di input all'interno di webapp, allo scopo di alterare la funzionalità originale della webapp attaccata. Si fa leva su una cattiva implementazione dei campi di input da parte del programmatore che non ha sanificato correttamente l'input dell'utente prima di processarlo.

Scope

Il target dei nostri test è come sempre la macchina metasploitable

- 192.168.50.101
- http://metasploitable

Inizieremo i nostri test settando la difficoltà a low per poi proseguire verso i livelli medium e high.

Tecnologie utilizzate

Per i nostri test utilizzeremo i seguenti software:

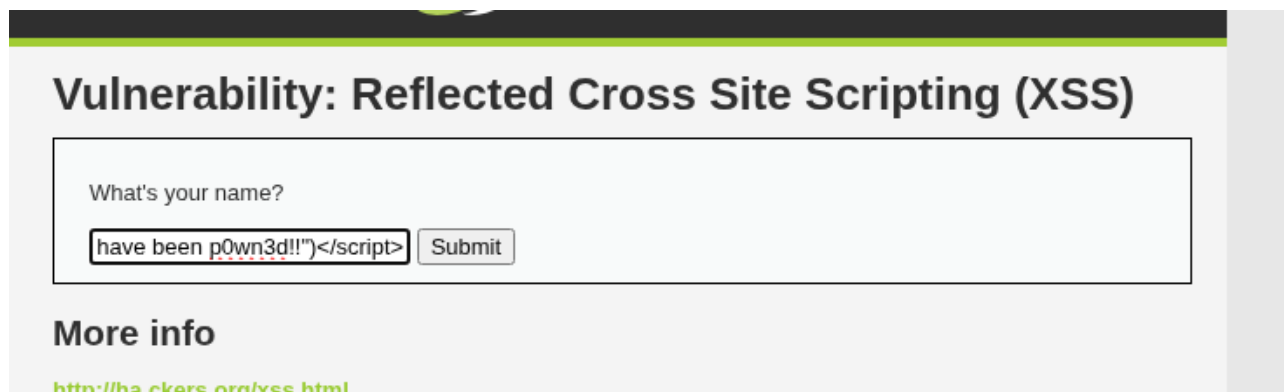
- BurpSuite v2025.7.4

Exploit

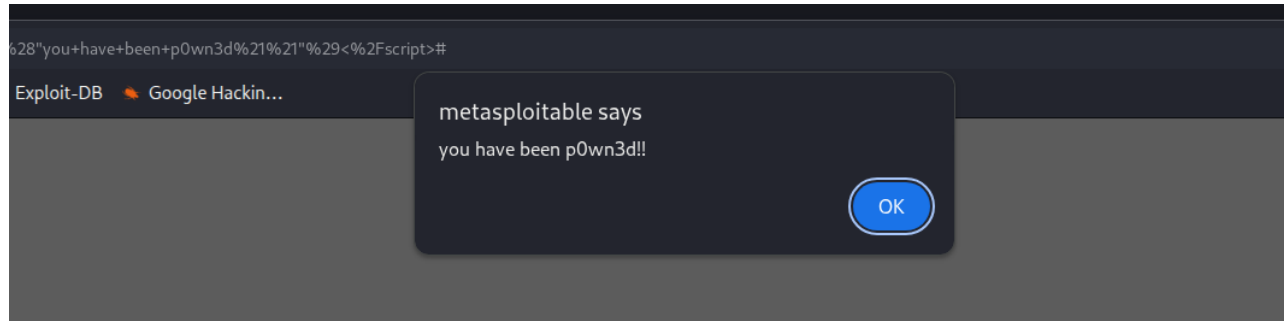
XSS - Security Low **done**

Per l'exploit di cross-site scripting introdurremo il seguente codice javascript nel campo di testo della DVWA:

```
<script>window.alert("you have been p0wn3d!!")</script>
```



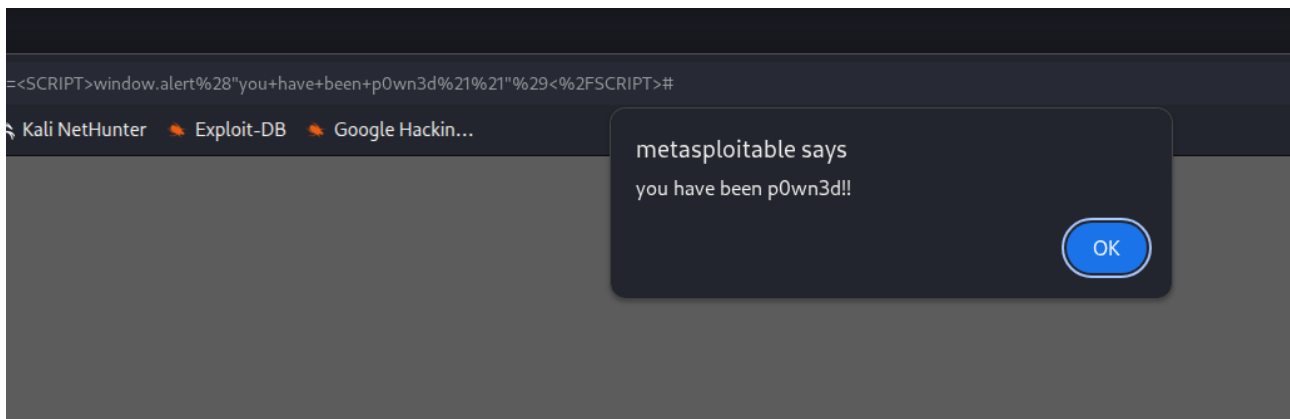
Una volta inserito il nostro codice, l'input viene passato alla pagina php che lo legge e lo esegue, in quanto non c'è nessun filtro che sanitizzi il testo inserito.



e il browser ci mostra il nostro messaggio minaccioso.

XSS - Security Medium **done**

Aumentando la sicurezza della DVWA a medium, la pagina filtra e rimuove il tag html `<script>`, quindi non possiamo più utilizzarlo, ma andando a guardare il codice noteremo che il filtro è case-sensitive, quindi se utilizziamo il tag `<SCRIPT>` con tutte lettere maiuscole, dovrebbe passare ugualmente.



Come vediamo dallo screenshot, il filtro funziona solo se il tag è scritto in minuscolo, che è la direttiva standard di html5, ma siccome il browser deve saper leggere anche altri dialetti (come *HTML4 Transitional* che è molto permissivo in termini di sintassi) e deve riuscire a rendere ugualmente le pagine anche se mal formattate, riesce a leggere tranquillamente il nostro codice.

XSS - Security High **done**